

VNBOT IMPLEMENTACIÓN

Fases, milestones, backlog, branches, releases y criterios de salida.

Transforma la visión y arquitectura de VNBOT en una secuencia concreta de trabajo. Define qué construir primero, qué dependencias existen, cómo comprobar cada fase, cómo organizar el repositorio, cómo gestionar releases y qué funcionalidades deben posponerse. La estrategia es incremental: cada fase deja un producto ejecutable, demostrable y verificable.

INCREMENTAL

11 PHASES

11 MILESTONES

RELEASE-READY

Tabla de Contenidos

1. Propósito	6
2. Estrategia general	6
2.1. Orden de prioridades	6
2.2. Principio de núcleo pequeño	6
2.3. Principio de una capacidad por vez	7
2.4. Definición de versión funcional	7
3. Fases generales	7
4. Fase 0 — Preparación del proyecto	8
4.1. Entregables	8
4.2. Tareas — Identidad	8
4.3. Tareas — Licencia	9
4.4. Tareas — Arquitectura (ADRs)	9
4.5. Tareas — Calidad	9
4.6. Tareas — Documentación y observabilidad	9
4.7. Criterio de salida	9
5. Fase 1 — Demo visual en GitHub Pages	10
5.1. Alcance	10
5.2. Tareas	10
5.3. Reglas	10
5.4. Criterio de salida	11

6. Fase 2 — Núcleo local	11
6.1. Alcance funcional	11
6.2. Implementación de almacenamiento	11
6.3. Memoria básica	12
6.4. Heurística	12
6.5. Recordatorios locales	12
6.6. Criterio de salida	12
7. Fase 3 — Backend privado	13
7.1. API	13
7.2. Persistencia	13
7.3. Redis y jobs	13
7.4. Docker	13
7.5. Notificaciones	14
7.6. Criterio de salida	14
8. Fase 4 — Inteligencia y audio	14
8.1. LLM Router	14
8.2. Clasificación	14
8.3. Búsqueda semántica	15
8.4. Audio	15
8.5. OCR posterior	15
8.6. Criterio de salida	15
9. Fase 5 — APK, desktop y CLI	15
9.1. APK	15

9.2. Desktop	16
9.3. CLI	16
9.4. Criterio de salida	16
10. Fase 6 — Grafo de memoria	16
10.1. Tareas	17
10.2. Graphify	17
10.3. Criterio de salida	17
11. Fase 7 — MCP Gateway	17
11.1. MCP interno	17
11.2. Gateway	18
11.3. Herramientas por riesgo	18
11.4. Criterio de salida	18
12. Fase 8 — Agentes y skills	18
12.1. Tareas	18
12.2. Skills iniciales	19
12.3. Criterio de salida	19
13. Fase 9 — Integraciones oficiales	19
13.1. Orden recomendado	19
13.2. Reglas	19
13.3. Criterio de salida	20
14. Fase 10 — Estabilización y comunidad	20
14.1. Seguridad	20
14.2. Rendimiento	20

14.3. Releases	20
14.4. Comunidad	21
15. Backlog priorizado	21
16. Milestones	21
17. Plan de branches y trabajo GitHub	22
17.1. Branches	22
17.2. Pull request	22
17.3. CI mínima	22
18. Gestión de assets visuales	23
18.1. Repositorio	23
18.2. Pipeline	23
18.3. Reglas	23
19. Seguridad durante el desarrollo	23
20. Definición de hecho	23
21. Pruebas por fase	24
21.1. Núcleo	24
21.2. Backend	24
21.3. IA	25
21.4. MCP	25
21.5. Plataformas	25
22. Criterios de release	25
23. Gestión de riesgos del plan	26

24. Primer sprint recomendado	26
24.1. Objetivo	26
24.2. Tareas	26
24.3. Resultado	27
25. Criterios de aceptación del plan	27
26. Conclusión	27
27. MVP comprimido: 0.1 unificado	28
27.1. Razón	28
27.2. Alcance del 0.1 unificado	28
27.3. Lo que NO incluye el 0.1	28
27.4. Criterio de salida del 0.1	29
28. Documentos adicionales de soporte	29

1. Propósito

Este documento transforma la visión y la arquitectura de VNBOT en una secuencia concreta de trabajo. Define qué construir primero, qué dependencias existen, cómo comprobar cada fase, cómo organizar el repositorio, cómo gestionar releases y qué funcionalidades deben posponerse para evitar que el proyecto se vuelva inmanejable.

El plan está diseñado para un proyecto open source que pueda evolucionar desde una demo estática hasta una plataforma distribuida con:

- Web/PWA, APK Android, Desktop, Docker y CLI.
- Memoria personal basada en grafo.
- Recordatorios confiables.
- Múltiples LLM con router y fallback.
- Skills y agentes personalizables.
- MCP para herramientas externas.
- Integraciones oficiales (calendario, Telegram, Gmail, etc.).

La estrategia es incremental: cada fase debe dejar un producto ejecutable, demostrable y verificable. No se debe esperar a tener todos los agentes, canales o integraciones para publicar una primera versión.

2. Estrategia general

2.1. Orden de prioridades

El orden de prioridades es deliberado y refleja las dependencias reales entre capacidades:

Confiabilidad de recordatorios
Memoria edita

2.2. Principio de núcleo pequeño

El núcleo inicial de VNBOT tendrá únicamente:

- Cuenta/modo local.
- Chat.
- Memoria.
- Grafo limitado.
- Recordatorios.
- Notificaciones básicas.
- Exportación.

- Diagnóstico.

El resto debe implementarse como módulos, adaptadores o plugins para evitar acoplamientos prematuros.

2.3. Principio de una capacidad por vez

No se implementará simultáneamente:

- Audio remoto.
- WhatsApp.
- Gmail.
- MCP externo.
- Multiagente.
- OCR.
- Sincronización.

Cada capacidad debe probarse individualmente antes de convertirse en una dependencia de otra.

2.4. Definición de versión funcional

Una versión puede publicarse cuando cumple 8 criterios:

- Se instala.
- Tiene documentación.
- Tiene migración si modifica datos.
- Tiene manejo de errores.
- Tiene pruebas automatizadas.
- Puede recuperarse de un fallo.
- No expone secretos.
- Tiene notas de release.

3. Fases generales

El plan se organiza en 11 fases secuenciales, cada una con un resultado verificable:

Fase	Nombre	Resultado
0	Preparación	Repositorio, decisiones, licencia y CI
1	Demo visual	GitHub Pages con experiencia simulada
2	Núcleo local	Memoria y recordatorios sin backend remoto

Fase	Nombre	Resultado
3	Backend privado	API, workers, scheduler y Docker
4	Inteligencia	LLM Router, búsqueda y audio
5	Plataformas	APK, desktop y CLI
6	Grafo	Memoria visual y relaciones avanzadas
7	MCP	Gateway, permisos y Graphify
8	Agentes	Skills, mascotas y autonomía limitada
9	Integraciones	Calendario, Telegram, Gmail y otros
10	Estabilización	Seguridad, rendimiento, releases y comunidad

4. Fase 0 — Preparación del proyecto

Crear una base de repositorio que permita trabajar de forma ordenada antes de desarrollar funciones complejas. Esta fase no produce código de producto, pero sin ella el resto se vuelve caótico.

4.1. Entregables

- Repositorio GitHub vnbot.
- Licencia MIT.
- README principal.
- CONTRIBUTING.md, SECURITY.md, CODE_OF_CONDUCT.md, CHANGELOG.md.
- THIRD_PARTY_NOTICES.md.
- Estructura monorepo.
- CI inicial.
- ADRs técnicos.
- Inventario de assets.

4.2. Tareas — Identidad

- [] Definir nombre oficial: VNBOT.
- [] Definir logo y wordmark.
- [] Reservar nombres de paquetes.
- [] Crear guía de nomenclatura.
- [] Migrar referencias antiguas de MinBot-Task.

4.3. Tareas — Licencia

- [] Añadir MIT para código.
- [] Documentar licencia de documentación.
- [] Documentar licencia de assets.
- [] Auditar repositorios de terceros.
- [] Crear inventario de modelos y datasets visuales.

4.4. Tareas — Arquitectura (ADRs)

- [] Crear ADR sobre frontend desacoplado.
- [] Crear ADR sobre SQLite/PostgreSQL.
- [] Crear ADR sobre workers.
- [] Crear ADR sobre MCP.
- [] Crear ADR sobre cifrado y privacidad.
- [] Crear ADR sobre distribución.

4.5. Tareas — Calidad

- [] Configurar formatter, linter y type checking.
- [] Configurar tests y pre-commit.
- [] Configurar Gitleaks y Semgrep.
- [] Configurar Trivy para imágenes.

4.6. Tareas — Documentación y observabilidad

- [] Crear documento de estrategia de sincronización (11-ESTRATEGIA-SYNC-VNBOT.md).
- [] Crear documento de testing y observabilidad (12-TESTING-Y-OBSERVABILIDAD-VNBOT.md).
- [] Crear documento de gobernanza (13-GOBERNANZA-DE-PROYECTO-VNBOT.md).
- [] Configurar axe-core en CI para accesibilidad.
- [] Configurar OpenTelemetry SDK en el proyecto base.
- [] Definir cobertura mínima de tests por módulo.
- [] Crear benchmarks automatizados para el grafo.

4.7. Criterio de salida

Un contribuidor nuevo puede clonar el repositorio, instalar dependencias, ejecutar una comprobación básica y comprender la arquitectura leyendo el README.

5. Fase 1 — Demo visual en GitHub Pages

Publicar una demostración navegable del concepto sin backend, secretos ni datos reales. Esta demo sirve como referencia visual y de marketing open source.

5.1. Alcance

- Landing.
- Onboarding simulado.
- Panel Hoy.
- Chat mock.
- Grafo con fixtures.
- Selector de agentes.
- Mascota principal y estados visuales.
- Vista de instalación.
- Links a GitHub, Docker y Releases.

5.2. Tareas

- [] Crear app Vite/React.
- [] Configurar base path de GitHub Pages.
- [] Implementar routing estático.
- [] Crear fixtures anónimos.
- [] Crear chat simulado y tarjetas de acción.
- [] Crear grafo demo.
- [] Añadir mascot states.
- [] Añadir responsive, high contrast y reduced motion.
- [] Configurar GitHub Action de deploy.

5.3. Reglas

- Ninguna API key.
- Ningún correo real.
- Ningún dato del usuario guardado en un servidor.
- Debe indicar qué partes son simuladas.
- Debe poder cargar sin backend.

5.4. Criterio de salida

La persona que visita la página entiende qué es VNBOT, ve el flujo principal y puede navegar por una experiencia consistente sin instalar nada.

6. Fase 2 — Núcleo local

Construir una versión utilizable para una sola persona, sin depender de un backend remoto ni de un proveedor LLM. Esta fase prueba que el ciclo Capturar→Interpretar→Confirmar→Ejecutar→Auditar funciona de punta a punta.

6.1. Alcance funcional

- Bóveda local.
- IndexedDB en PWA.
- SQLite en desktop/local.
- Chat.
- Heurística.
- Memorias, nodos y aristas básicos.
- Recordatorios locales.
- Notificaciones disponibles en plataforma.
- Exportación/importación.

6.2. Implementación de almacenamiento

PWA (IndexedDB):

- [] Crear esquema IndexedDB.
- [] Añadir migraciones de cliente.
- [] Crear repositorios.
- [] Crear índices por fecha/tipo.
- [] Separar plaintext en memoria de UI.
- [] Añadir expiración de datos temporales.

Local/desktop (SQLite):

- [] Crear SQLite adapter.
- [] Crear migraciones Alembic o equivalente.
- [] Configurar backup.
- [] Crear bloqueo de archivo.
- [] Crear comando vnbot doctor.

6.3. Memoria básica

- [] Crear MemoryNode y MemoryEdge.
- [] Añadir procedencia, confianza y sensibilidad.
- [] Implementar CRUD y búsqueda textual.
- [] Implementar olvidar y exportación.

6.4. Heurística

Debe detectar al menos los siguientes patrones de intención:

- "Recuérdame".
- "Avísame".
- "Tengo que".
- "Necesito".
- "Guarda que".
- "Olvida que".
- "Añade a la lista".
- Fechas relativas comunes.
- Horas básicas.

La heurística debe indicar baja confianza cuando no pueda resolver una instrucción con seguridad. No debe inventar interpretaciones.

6.5. Recordatorios locales

- [] Crear entidad Reminder y Occurrence.
- [] Resolver timezones.
- [] Implementar recurrencia.
- [] Implementar posponer, completar y cancelación.
- [] Evitar duplicados.
- [] Crear historial local.

6.6. Criterio de salida

El usuario puede crear una memoria y un recordatorio, cerrar la aplicación, abrirla de nuevo, recuperar los datos y exportarlos sin conexión.

7. Fase 3 — Backend privado

Convertir el núcleo local en una plataforma servidor con API, autenticación, workers, scheduler y Docker. Esta fase permite multi-dispositivo y prepara la base para escalado horizontal posterior.

7.1. API

- [] Crear FastAPI.
- [] Crear /api/v1.
- [] Crear schemas Pydantic.
- [] Crear manejo de errores.
- [] Crear request_id/trace_id.
- [] Crear OpenAPI.
- [] Crear autenticación.
- [] Crear autorización por workspace.

7.2. Persistencia

- [] Crear SQLAlchemy y Alembic.
- [] Validar SQLite y PostgreSQL.
- [] Crear índices y transacciones.
- [] Crear soft delete.
- [] Crear backups.

7.3. Redis y jobs

- [] Añadir Redis.
- [] Definir job schema.
- [] Crear worker y scheduler.
- [] Crear locks.
- [] Crear reintentos y dead-letter queue.
- [] Crear endpoint de estado de jobs.

7.4. Docker

- [] Dockerfile API, worker y scheduler.
- [] Compose local y compose server.
- [] .env.example.
- [] Healthchecks y volúmenes.

- [] Usuario no root.

7.5. Notificaciones

- [] Notificación web.
- [] Notificación desktop.
- [] Notificación local Android posteriormente.
- [] Estado de delivery.
- [] Reintentos.
- [] Preferencias de usuario.

7.6. Criterio de salida

Una instalación Docker puede registrar un usuario, guardar memorias, crear recordatorios, procesar jobs, reiniciarse y conservar el estado.

8. Fase 4 — Inteligencia y audio

Añadir IA configurable sin convertirla en una dependencia obligatoria ni permitir que actúe sin validación. El LLM es un intérprete, no un decisor.

8.1. LLM Router

- [] Definir interfaz común.
- [] Implementar Ollama.
- [] Implementar OpenAI-compatible.
- [] Implementar proveedor adicional.
- [] Añadir structured outputs.
- [] Añadir fallback, timeouts y circuit breaker.
- [] Añadir presupuesto.
- [] Añadir conteo de tokens agregado.

8.2. Clasificación

- [] Intent classifier.
- [] Extracción de fecha y entidades.
- [] Selección de skill.
- [] Confidence score.
- [] Detección de ambigüedad.
- [] Validación Pydantic.

8.3. Búsqueda semántica

- [] Elegir embedding local inicial.
- [] Crear índice.
- [] Asociar vector a nodo.
- [] Crear búsqueda híbrida.
- [] Añadir filtros.
- [] Invalidar embeddings al editar/borrar.
- [] Añadir fuentes a respuesta.

8.4. Audio

- [] UI de grabación.
- [] Endpoint temporal.
- [] Validación de audio.
- [] Job de transcripción.
- [] Whisper local.
- [] Revisión de transcript.
- [] Retención configurable.
- [] Borrado seguro.

8.5. OCR posterior

- [] Subida de imagen.
- [] Job OCR.
- [] Preview de extracción.
- [] Corrección manual.
- [] Conversión a memoria o recordatorio.

8.6. Criterio de salida

El sistema puede interpretar instrucciones con un modelo local o externo, mostrar una propuesta estructurada, pedir aclaraciones y ejecutar solo después de validarla.

9. Fase 5 — APK, desktop y CLI

9.1. APK

- [] Integrar PWA con Capacitor.

- [] Permisos de micrófono.
- [] Notificaciones locales.
- [] Estado de red.
- [] Filesystem local.
- [] Manejo de suspensión.
- [] Generar APK debug y release.
- [] Firmar APK.
- [] Pruebas en dispositivos reales.

9.2. Desktop

- [] Crear shell Tauri.
- [] Integrar frontend.
- [] Configurar SQLite local.
- [] Notificaciones nativas.
- [] Acceso a archivos autorizado.
- [] Auto-lock.
- [] Build Windows, Linux y macOS.
- [] Actualización firmada posteriormente.

9.3. CLI

- [] init, doctor, health.
- [] add, search, reminders.
- [] backup, restore.
- [] mcp.
- [] Salida JSON para automatizaciones.
- [] No mostrar secretos en argumentos.

9.4. Criterio de salida

El mismo usuario puede acceder a sus memorias y recordatorios desde web, APK o desktop, respetando el modelo de permisos y sincronización definido.

10. Fase 6 — Grafo de memoria

Convertir la memoria básica en una representación relacional visible y útil. El grafo permite descubrir conexiones que la búsqueda textual no encuentra.

10.1. Tareas

- [] Definir catálogo de tipos de nodo.
- [] Definir catálogo de relaciones.
- [] Crear resolución de entidades.
- [] Detectar duplicados.
- [] Crear búsqueda por relaciones.
- [] Añadir profundidad configurable.
- [] Crear grafo limitado.
- [] Añadir inspector.
- [] Crear modo lista.
- [] Añadir export JSON.
- [] Añadir corrección de aristas.
- [] Añadir expiración temporal.
- [] Añadir contradicciones.

10.2. Graphify

- [] Crear adaptador MCP.
- [] Registrar servidor.
- [] Mostrar tools/resources.
- [] Permitir solo graph.read al principio.
- [] Crear referencias cruzadas.
- [] No mezclar automáticamente datos personales.
- [] Crear opción de desconexión.

10.3. Criterio de salida

El usuario puede buscar una entidad, ver sus relaciones relevantes, abrir la memoria origen, corregir una relación y eliminarla sin afectar datos no relacionados.

11. Fase 7 — MCP Gateway

Crear una capa segura para conectar herramientas externas y exponer herramientas internas. MCP es el sistema de plugins con permisos granulares.

11.1. MCP interno

- [] memory_search, memory_create, memory_update, memory_forget.

- [] graph_expand.
- [] reminder_create, reminder_complete, reminder_snooze.
- [] list_manage.

11.2. Gateway

- [] Registro de servidor.
- [] Transporte stdio y Streamable HTTP.
- [] Handshake y descubrimiento.
- [] Scopes y policy engine.
- [] Timeouts y rate limit.
- [] Healthcheck.
- [] Auditoría.
- [] Revocación.

11.3. Herramientas por riesgo

Nivel	Ejemplos
Bajo	Lectura de memoria autorizada, consulta de estado, búsqueda de nodos.
Medio	Crear memoria, crear recordatorio, crear evento.
Alto	Borrador de email, lectura de filesystem, compartir contenido.
Crítico	Enviar email, escribir archivos, eliminar datos masivamente, acciones financieras (siempre fuera del MVP).

11.4. Criterio de salida

Un servidor MCP puede conectarse, mostrar sus capacidades, recibir solo los scopes elegidos y ejecutar una herramienta con confirmación y auditoría.

12. Fase 8 — Agentes y skills

Permitir personalización avanzada sin convertir las instrucciones en código arbitrario. Los agentes son configuraciones, no scripts.

12.1. Tareas

- [] Crear Agent entity y Skill manifest.
- [] Crear selector de modelo, memoria y herramientas.
- [] Crear niveles de autonomía.

- [] Crear presupuesto por agente.
- [] Crear modo simulación.
- [] Crear auditoría por agente.
- [] Vincular mascota a agente.
- [] Implementar skills base.

12.2. Skills iniciales

```
memory.save<br/>memory.search<br/>memory.correct<br/>memory.forget<br/>reminder.create<br/>reminder.sn
```

12.3. Criterio de salida

El usuario puede crear un agente, asignarle skills y herramientas, revisar permisos, ejecutar una prueba controlada y revocarlo.

13. Fase 9 — Integraciones oficiales

13.1. Orden recomendado

01. Calendario interno.
02. Google Calendar.
03. Telegram Bot API.
04. Gmail lectura/borradores.
05. Outlook.
06. WhatsApp Business Cloud API.
07. Discord bot oficial.

13.2. Reglas

Cada integración debe documentar:

- API oficial.
- Scopes.
- Datos leídos.
- Datos escritos.
- Costes.
- Límites.
- ToS.

- Revocación.
- Healthcheck.
- Fallback.

13.3. Criterio de salida

La integración puede activarse, probarse, revocarse y operar sin que un fallo externo rompa el núcleo de VNBOT.

14. Fase 10 — Estabilización y comunidad

14.1. Seguridad

- [] Threat model actualizado.
- [] Pentest básico.
- [] SAST y DAST.
- [] Secret scanning.
- [] Escaneo de imágenes.
- [] Revisión MCP, archivos y backups.

14.2. Rendimiento

- [] Pruebas de 10.000 memorias.
- [] Pruebas de 1.000 recordatorios.
- [] Pruebas de workers.
- [] Pruebas de reconexión.
- [] Pruebas de grafo.
- [] Pruebas en Android de gama baja.
- [] Pruebas de consumo desktop.

14.3. Releases

- [] Versionado semántico.
- [] Checksums.
- [] SBOM.
- [] Firma de artefactos.
- [] Release notes.
- [] Migraciones.
- [] Canal beta y canal stable.

14.4. Comunidad

- [] Guía de contribución.
- [] Issue templates y PR template.
- [] CODEOWNERS.
- [] Documentación de plugins.
- [] Roadmap público.
- [] Inventario de assets.
- [] Política de seguridad.

15. Backlog priorizado

El backlog se organiza en 5 prioridades que mapean a las fases:

Prioridad	Nombre	Items principales
P0	Bloqueante	Repositorio y licencia, modelo de dominio, memoria CRUD, recordatorio puntual, recurrencia, zona horaria, scheduler, idempotencia, notificación local, exportación, heurística, healthchecks, CI.
P1	MVP público	Grafo limitado, búsqueda híbrida, PostgreSQL, Redis, worker, Docker, LLM Router, PWA, demo GitHub Pages, logs y auditoría.
P2	Beta multiplataforma	APK, desktop, CLI, audio, embeddings locales, mascotas por agente, MCP interno, Graphify read-only.
P3	Plataforma extensible	MCP externo completo, skills, agentes personalizados, Google Calendar, Telegram, Gmail borradores, sincronización, workspaces compartidos.
P4	Escala y comunidad	Marketplace de skills, plugin SDK, observabilidad avanzada, Kubernetes, multi-región, agentes coordinados.

16. Milestones

Cada fase tiene un milestone con salida verificable:

Milestone	Nombre	Salida
M0	Repositorio listo	Estructura, licencia, CI, documentación y ADRs.
M1	Demo pública	GitHub Pages navegable con chat, panel y grafo mock.
M2	Local usable	Memoria y recordatorios offline.
M3	Servidor privado	Docker con API, database, worker y scheduler.

Milestone	Nombre	Salida
M4	Inteligencia controlada	LLM Router, structured outputs y búsqueda semántica.
M5	Multiplataforma	APK y desktop instalables.
M6	Grafo real	Relaciones, procedencia, filtros y exportación.
M7	MCP seguro	Gateway, scopes, Graphify opcional y auditoría.
M8	Agentes	Skills, mascotas y autonomía limitada.
M9	Integraciones	Calendario, Telegram y correo limitado.
M10	Release estable	Seguridad, performance, backups, SBOM y documentación.

17. Plan de branches y trabajo GitHub

17.1. Branches

main → estable
next → integración de próxima versión
feature/* → funcionalidad
develop → desarrollo

17.2. Pull request

Cada PR debe incluir:

- Problema.
- Solución.
- Archivos afectados.
- Tests.
- Capturas si modifica UI.
- Migración si modifica datos.
- Cambios de seguridad.
- Impacto en compatibilidad.

17.3. CI mínima

- Lint.
- Type check.
- Unit tests.
- Integration tests.
- Build web.
- Build API.
- SAST.

- Secret scan.
- Dependency audit.

18. Gestión de assets visuales

18.1. Repositorio

assets/
|— mascot/
|— agents/
|— sprites/
|— hud/
|— icons/
|— palettes/

18.2. Pipeline

Generación conceptual
→ selección
→ limpieza manual
→ palette lock
→ sprite sheet

18.3. Reglas

- No incluir watermarks.
- No utilizar assets de stock sin licencia.
- Guardar origen y modelo utilizado.
- Registrar licencia.
- Probar en 16/32/64/128 px.
- Mantener escalado nearest-neighbor.

19. Seguridad durante el desarrollo

- Nunca subir .env.
- Ejecutar Gitleaks en cada PR.
- No usar datos reales en fixtures.
- Usar cuentas sandbox para integraciones.
- Rotar tokens de desarrollo.
- No incluir audios reales en tests.
- Mockear proveedores LLM.
- Crear MCP malicioso de prueba.
- Verificar aislamiento entre workspaces.
- Revisar dependencias nuevas.

20. Definición de hecho

Una tarea no está terminada solo porque funciona localmente. Debe cumplir 12 criterios:

01. Código implementado.
02. Tests.
03. Manejo de error.
04. Estados de loading/offline.
05. Permisos.
06. Auditoría.
07. Documentación.
08. Compatibilidad definida.
09. Migración si aplica.
10. Revisión de seguridad.
11. Capturas o pruebas visuales si aplica.
12. Criterio de rollback.

21. Pruebas por fase

Las pruebas se organizan por área y cubren escenarios críticos de cada fase:

21.1. Núcleo

- Fecha ambigua.
- Zona horaria.
- Recurrencia.
- Cierre inesperado.
- Borrado.
- Exportación.

21.2. Backend

- Reinicio API.
- Reinicio worker.
- Duplicación de jobs.
- Redis caído.
- Base de datos lenta.
- Delivery fallido.

21.3. IA

- JSON inválido.
- Prompt injection.
- LLM caído.
- Rate limit.
- Fallback heurístico.
- Memoria no autorizada.

21.4. MCP

- Tool timeout.
- Scope insuficiente.
- Servidor malicioso.
- Revocación.
- Respuesta demasiado grande.
- Replay.

21.5. Plataformas

- APK sin permiso.
- Desktop sin conexión.
- PWA offline.
- Escalado pixelado.
- Reduced motion.
- Diferentes tamaños de pantalla.

22. Criterios de release

Canal	Características
Alpha	Puede romperse, datos de prueba, sin promesa de compatibilidad, logs ampliados.
Beta	Flujo núcleo estable, migraciones, backups, instaladores, reporte de errores.
Stable	Suite de pruebas, seguridad revisada, documentación, compatibilidad definida, restore probado, releases firmados o con checksums.

23. Gestión de riesgos del plan

Cada riesgo tiene una señal temprana detectable y una acción correctiva concreta:

Riesgo	Señal temprana	Acción
Scope excesivo	Muchas features P0	Congelar alcance
Backend monolítico	UI llama directamente a SDKs	Mover a adaptadores
Scheduler poco fiable	Duplicados	Idempotencia y locks
Coste LLM	Muchas llamadas por chat	Router y presupuesto
MCP inseguro	Tools sin scopes	Policy engine obligatorio
UI pesada	FPS bajo en móvil	Sprite sheets y fallback
Licencias inciertas	Assets sin origen	Retirar y auditar
Comunidad bloqueada	PRs difíciles de probar	CI y docs
Pérdida de datos	Backups nunca restaurados	Prueba periódica

24. Primer sprint recomendado

24.1. Objetivo

Crear la primera rebanada vertical completa que demuestre el ciclo central de VNBOT:

```
Chat<br/>→ interpretar recordatorio<br/>→ confirmar<br/>→ guardar<br/>→ programar<br/>→ notificar<br/>
```

24.2. Tareas

- ☐ Crear repositorio base.
- ☐ Crear modelo Reminder.
- ☐ Crear parser heurístico.
- ☐ Crear endpoint de propuesta.
- ☐ Crear confirmación.
- ☐ Crear SQLite.
- ☐ Crear scheduler local.
- ☐ Crear notificación mock.
- ☐ Crear panel Hoy mínimo.
- ☐ Crear test E2E.
- ☐ Crear estado visual del golem.

24.3. Resultado

No se busca todavía una aplicación completa. Se busca demostrar que el ciclo más importante funciona de punta a punta sin perder datos ni confundir al usuario.

25. Criterios de aceptación del plan

El plan se considera suficientemente definido cuando:

- Cada módulo tiene una fase.
- Cada fase tiene entregables.
- Cada entregable tiene criterio de salida.
- Las dependencias están identificadas.
- El MVP tiene un backlog P0/P1.
- La distribución está contemplada desde el inicio.
- La seguridad aparece en cada fase.
- La UI y el backend tienen estados compatibles.
- Las tareas asíncronas tienen reintentos.
- La comunidad puede contribuir sin conocer todo el sistema.

26. Conclusión

VNBOT debe implementarse como una secuencia de productos cada vez más capaces, no como un único lanzamiento gigantesco. La primera versión debe probar la confiabilidad del núcleo: capturar, estructurar, recordar y notificar.

Después se añaden IA, audio, plataformas, grafo, MCP, agentes e integraciones, siempre con interfaces estables, jobs auditables y permisos visibles.

La estrategia de implementación queda resumida así:

Primero confiabilidad.
Después memoria.
Luego distribución.
Después extensibilidad.
Fi

La autonomía avanzada solo será valiosa si el usuario puede saber qué ocurrió, por qué ocurrió, qué datos se utilizaron y cómo detener o revertir la acción.

27. MVP comprimido: 0.1 unificado

27.1. Razón

Las fases originales 0.1 (demo), 0.2 (núcleo local) y 0.3 (servidor) pueden fusionarse en un bloque que entregue valor real más rápido. Esto reduce el time-to-value sin sacrificar calidad.

27.2. Alcance del 0.1 unificado

- [] Monorepo con CI (lint, typecheck, tests, secret scan).
- [] Web/PWA con chat.
- [] Backend embebido o Docker con SQLite.
- [] Fallback heurístico (sin LLM) documentado y testeado.
- [] Al menos 1 proveedor LLM configurado.
- [] Memorias CRUD con búsqueda textual.
- [] Recordatorios puntuales y recurrentes con scheduler.
- [] Grafo básico (CRUD de nodos y edges).
- [] Exportación e importación cifrada.
- [] Panel de auditoría básico.
- [] OpenTelemetry tracing en endpoints principales.
- [] Tests unitarios con 60% cobertura en core.
- [] axe-core en CI sin violaciones AA críticas.
- [] Demo estática en GitHub Pages (paralelo, no bloqueante).

27.3. Lo que NO incluye el 0.1

- APK, Desktop nativo, CLI nativo.
- MCP externo.
- Búsqueda semántica (requiere embeddings).
- Múltiples usuarios/workspaces.
- Sincronización multi-dispositivo.
- Audio.
- Agentes personalizables.

27.4. Criterio de salida del 0.1

Una persona puede: crear una memoria, crear un recordatorio, buscar, exportar e importar — sin servidor externo. Los tests pasan en CI. El dashboard de observabilidad muestra métricas básicas. La documentación está actualizada.

28. Documentos adicionales de soporte

Los siguientes documentos deben existir antes de la implementación activa:

Documento	Propósito	Necesario antes de
11-ESTRATEGIA-SYNC-VNBOT.md	Diseño de sincronización multi-dispositivo	Fase 0.3 (plataformas)
12-TESTING-Y-OBSERVABILIDAD-VNBOT.md	Estrategia de tests y OpenTelemetry	Fase 0.1 (inicio de código)
13-GOBERNANZA-DE-PROYECTO-VNBOT.md	Modelo de gobernanza y contribución	Fase 0 (preparación repo)